



《AI大模型Ragent项目》——查询重写与语义增强机制

来自：拿个offer-开源&项目实战



马丁

2026年03月08日 19:39

上一篇讲了会话记忆，你的 RAG 系统终于能知道之前聊了什么了。用户先问“iPhone 16 Pro 的退货政策是什么”，再追问“那它的保修期呢”，模型看到了完整的对话历史，知道“它”指的是 iPhone 16 Pro，回答没问题。

但别高兴得太早。

RAG 系统在回答之前，要先去向量数据库检索相关的 chunk。检索系统拿到的 query 是什么？是用户的原始问题——那它的保修期呢。“它”这个字对检索系统来说没有任何语义信息。向量化之后，“那它的保修期呢”和“iPhone 16 Pro 保修期”的向量距离可能相差甚远。检索召回的结果大概率不是你想要的——可能是“笔记本电脑保修政策”“家电延保服务”这些不相关的内容。

问题出在哪？**模型有记忆，但检索系统没有。**

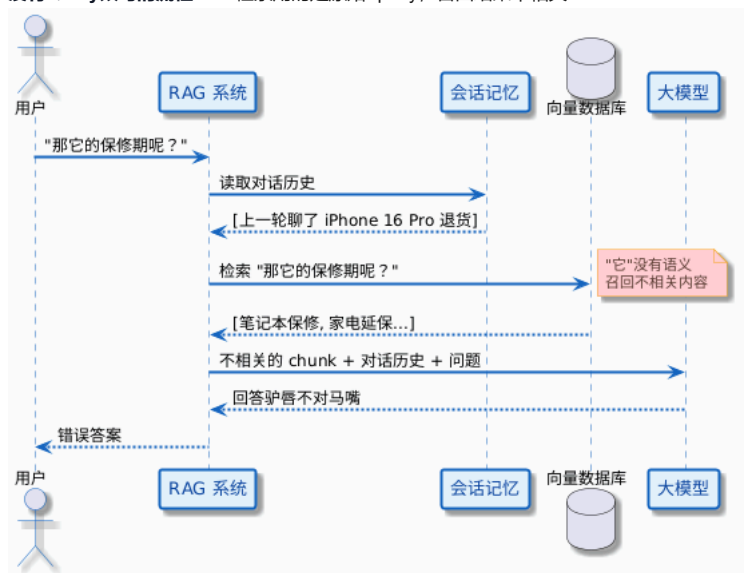
解法其实很直接：在检索之前，先把“那它的保修期呢”改写成“iPhone 16 Pro 的保修期”，再拿去检索。这就是今天要讲的 Query 改写（Query Rewrite）。

检索系统的“失忆”问题

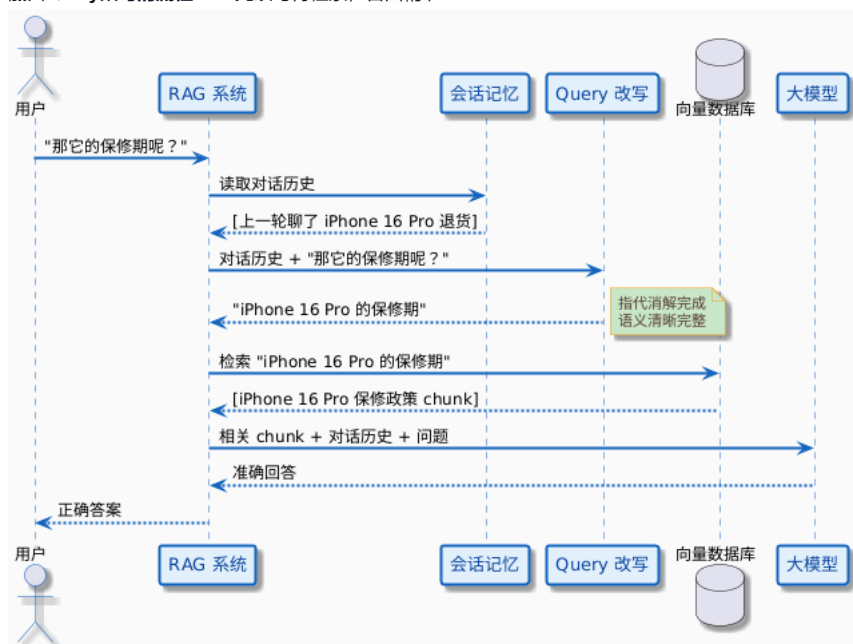
1. 模型有记忆，但检索没有

用一张图来看看问题出在哪。

没有Query改写的流程——检索用的是原始 query，召回结果不相关：



加入Query改写的流程——先改写再检索，召回精准：



区别就在中间多了一步 Query 改写。这一步把用户含糊的追问转化成了一个清晰、完整、独立的检索查询。

2. 不只是“它”的问题

指代消解（把“它”替换成具体实体）是最常被提到的改写场景，但 Query 改写要解决的问题远不止这个。

看几个电商客服场景下的典型问题：

省略上下文

用户在聊了几轮 iPhone 16 Pro 之后，突然问“还有别的颜色吗？”。别的颜色是什么产品的？检索系统不知道。如果直接拿“还有别的颜色吗”去检索，可能召回所有产品的颜色信息，而不是 iPhone 16 Pro 的。

口语化表达

用户说“东西坏了咋整？”。知识库里的文档标题大概率是产品故障维修流程或售后服务指南，不会写东西坏了咋整。口语化的 query 和正式文档之间存在语义鸿沟，检索效果打折扣。

多意图混合

用户问“退货流程是什么，运费谁承担？”。这一句话里其实包含两个独立的问题：退货流程和运费承担方。一次检索很难同时命中两个主题的 chunk。

模糊描述

用户说“那个很贵的手机”。哪个？多少钱算贵？检索系统没有上下文，无法理解这种模糊描述。

这些问题的共同点是：**用户的原始query对检索系统不够友好**。Query 改写要做的事情，就是在检索之前把原始 query 转化为一个独立的、完整的、对检索系统友好的查询。

Query 改写的五种策略

1. 指代消解（Coreference Resolution）

指代消解，说白了就是把代词替换成它指代的具体实体。这是多轮对话中最高频的改写场景。

常见的代词和指代表达：

代词 / 指代表达	示例	改写结果
它、它的	那它的保修期呢？	iPhone 16 Pro 的保修期
这个、那个	这个支持分期吗？	iPhone 16 Pro 支持分期吗？
上面说的	上面说的退货条件再详细说说	iPhone 16 Pro 拆封后退货条件的详细说明
同样的问题	另一款也是这样吗？	iPhone 16 Plus 的退货政策和 iPhone 16 Pro 一样吗？

指代消解的关键在于：**你必须结合对话历史才能确定代词指的是什么**。脱离对话历史，“它”可以是任何东西。

需要注意一个边界情况：有时候“它”的指代并不明确。比如用户前面同时聊了 iPhone 16 Pro 和 AirPods Pro，然后问“它的保修期呢”，“它”到底指哪个？这种情况下，改写模型需要根据最近的上下文做判断——通常取最近一次被提到的实体。

2. 上下文补全（Context Completion）

人在多轮对话中会自然地省略信息，因为他觉得对方应该知道上下文。但检索系统不知道。

原始 query	省略了什么	改写结果
还有别的颜色吗？	什么产品的颜色	iPhone 16 Pro 还有其他颜色可选吗？
价格呢？	什么东西的价格	iPhone 16 Pro 256GB 白色钛金属的价格是多少？
能退吗？	什么产品、什么情况下退	iPhone 16 Pro 拆封后能退货吗？
多久能到？	什么产品发货到哪里	iPhone 16 Pro 下单后多久能送到？

上下文补全和指代消解经常同时出现。“价格呢”既省略了产品名（上下文补全），又省略了主语（可以理解为“它的价格呢”，指代消解）。实际改写时，大模型会一并处理，不需要你单独区分。

3. 口语化转正式（Colloquial to Formal）

用户的提问方式和知识库里的文档写法通常差异很大。用户说人话，文档写书面语。

口语 query	知识库中的正式表达	改写结果
东西坏了咋整	产品故障报修流程	产品故障后的维修和报修流程
快递咋还没到	订单物流查询 / 发货时效	订单发货后物流状态查询
能不能便宜点	优惠活动 / 促销政策 / 折扣信息	当前可用的优惠活动和折扣信息

买贵了能补差价不	价格保护政策	商品降价后是否支持差价补偿
----------	--------	---------------

这种改写有一个特点：**不依赖对话历史**。即使是第一轮对话，口语化的 query 也需要转化成更正式的表达。所以口语化转正式其实在单轮对话的 RAG 中也有价值。

不过要注意，口语化转正式不是“翻译”，而是“意图提取”。“能不能便宜点”的意图不是字面上的“降低价格”，而是“查询有没有优惠”。改写模型需要理解用户的真实意图。

4. 多意图拆分 (Intent Decomposition)

用户有时候一句话里包含多个问题：

退货流程是什么，运费谁承担？	→ 两个独立意图
iPhone 16 Pro 和 iPhone 16 Plus 有什么区别？	→ 一个对比意图，不需要拆
我想退货，另外帮我查一下保修期	→ 两个完全不相关的意图

拆分后，每个子查询分别去检索，各自召回最相关的 chunk，合并后再生成答案。

不是所有长 query 都需要拆。“iPhone 16 Pro 的价格和颜色”虽然问了两个方面，但通常在同一个产品介绍 chunk 里就能找到，不需要拆成两次检索。拆分的判断标准是：两个意图是否可能分布在不同的 chunk 里。

多意图拆分的实现复杂度比前面几种策略高，而且拆分后每个子查询都要走一遍检索流程，成本翻倍。在实际项目中，如果业务场景下多意图问题不多，可以先不实现这个策略。

5. 关键词扩展 (Keyword Expansion)

关键词扩展是补充同义词和相关术语，提高检索的召回率。

原始 query	扩展后
七天无理由退货	七天无理由退货 退换货政策 无条件退款 退货期限
屏幕碎了	屏幕碎裂 屏幕破损 屏幕维修 碎屏险
充不进去电	无法充电 充电故障 充电接口问题 电池问题

这种改写主要对**关键词检索 (BM25)**有帮助——BM25 是按词匹配的，同义词扩展能提高命中率。对向量检索来说，帮助有限，因为向量检索本身就能理解语义相似性（屏幕碎了和屏幕破损的向量已经很接近了）。

如果你的 RAG 系统用的是混合检索（向量 + BM25），关键词扩展在 BM25 那一路上会有明显提升。

6. 五种策略对比

策略	解决的问题	依赖对话历史	实现复杂度	对检索的影响	适用场景
指代消解	代词无法检索	是	低	必需，否则检索失败	多轮对话（最常用）
上下文补全	省略信息无法检索	是	低	必需，否则检索不精准	多轮对话
口语化转正式	口语与文档的语义鸿沟	否	中	有提升，尤其对 BM25	单轮 + 多轮
多意图拆分	一次检索无法覆盖多个意图	否	高	有提升，但成本翻倍	复杂咨询场景
关键词扩展	同义词不匹配	否	低	对 BM25 有帮助	混合检索场景

实际项目中，指代消解和上下文补全是必做的（不说的话多轮对话检索基本不可用）。口语化转正式建议做（投入产出比高）。多意图拆分和关键词扩展根据业务需要决定。

用大模型做 Query 改写

前面讲了五种改写策略，但在实际实现中，你不需要为每种策略单独写一套规则。用大模型做改写，一个 Prompt 就能覆盖大部分场景——指代消解、上下文补全、口语化转正式，大模型一次性搞定。

1. 改写 Prompt 的设计

1.1 基础版改写 Prompt

这个 Prompt 适合大多数场景，简单直接：

你是一个查询改写助手。根据对话历史和用户的最新问题，将问题改写为一个独立的、完整的检索查询。

要求：

- 如果最新问题中包含代词（它、这个、那个等）或省略了关键信息，请结合对话历史补全
- 如果问题已经足够完整清晰，请原样输出，不要画蛇添足
- 只输出改写后的查询，不要输出任何解释、前缀或多余内容
- 改写后的查询应该是一个独立的句子，不依赖对话历史也能理解

对话历史：

{history}

用户最新问题：{query}

改写后的查询：

看几个改写效果：

对话历史	原始 query	改写结果
用户问了 iPhone 16 Pro 退货政策	那它的保修期呢？	iPhone 16 Pro 的保修期是多久？
用户在聊 AirPods Pro 的颜色	价格呢？	AirPods Pro 的价格是多少？
无历史（第一轮）	东西坏了咋整？	产品故障后的维修流程
用户在聊 iPhone 16 Pro	还有别的吗？	iPhone 16 Pro 还有其他配置或颜色可选吗？

基础版 Prompt 能很好地处理指代消解和上下文补全，对口语化转正式也有一定效果。

1.2 进阶版改写 Prompt

如果你的业务场景需要支持多意图拆分，可以用进阶版 Prompt。输出格式改为 JSON，方便程序解析：

你是一个查询改写助手。根据对话历史和用户的最新问题，将问题改写为适合检索的查询。

要求：

- 补全代词和省略的上下文信息
- 将口语化表达转化为更正式、更适合检索的表达
- 如果问题包含多个独立意图，拆分为多个子查询
- 如果问题已经完整清晰且只有一个意图，只输出一个查询
- 以 JSON 格式输出，格式为：{"queries": ["查询1", "查询2"]}
- 不要输出 JSON 以外的任何内容

对话历史：

{history}

用户最新问题：{query}

进阶版的输出示例：

原始 query	输出 JSON
那它的保修期呢？	{"queries": ["iPhone 16 Pro 的保修期是多久"]}
退货流程和运费谁承担？	{"queries": ["退货流程是什么", "退货运费由谁承担"]}
东西坏了咋整？	{"queries": ["产品故障后的维修和报修流程"]}

基础版够用就用基础版。进阶版虽然功能更强，但 JSON 解析增加了复杂度，模型偶尔也会输出格式不规范的 JSON。除非你的业务确实有多意图拆分的需求，否则基础版是更稳的选择。

2. 改写效果的好坏取决于什么

改写质量受几个因素影响：

对话历史的质量：如果对话历史被摘要压缩得太狠，关键实体可能已经丢了。比如摘要里只写了“客户咨询手机售后”，没有提到具体型号，改写时就无法把“它”替换成具体产品名。所以会话记忆的摘要质量直接影响改写质量。

Prompt的设计：Prompt 里的规则要明确——什么时候改写、什么时候原样输出。如果 Prompt 没有说“已经完整的 query 不需要改写”，模型可能会画蛇添足，把一个本来就很好的 query 改得面目全非。

模型的能力：小模型在复杂指代消解（多个实体交替出现）和口语化理解上可能不够准确。改写任务对模型要求不高，Qwen2.5-7B-Instruct 级别的模型就能胜任大部分场景。

常见的改写失败案例：

失败类型	原始 query	错误改写	问题原因
过度改写	它多少钱？	iPhone 16 Pro 256GB 沙漠色钛金属版在京东平台的售价是多少？	用户没提过平台和颜色，模型自己加的
改写不足	那个也是这个价吗？	那个也是这个价吗？	“那个”和“这个”都没有替换
偏离原意	能不能便宜点？	如何投诉商品定价过高？	用户想问优惠，不是投诉

遇到这类问题，通常通过调整 Prompt 来解决。比如加一条“不要添加用户没有提到的信息”可以抑制过度改写。

3. 改写的成本与时机

每次 Query 改写都要调一次大模型 API，增加大约 200~500ms 的延迟和一小笔 Token 费用。虽然单次成本不高，但并不是每次请求都需要改写。

什么时候可以跳过改写：

- 第一轮对话**：没有对话历史，不存在指代消解和上下文补全的需求。如果 query 本身够完整（iPhone 16 Pro 的退货政策是什么），直接检索就行
- query本身已经完整**：用户的问题明确包含了主体、动作和对象，不包含代词和省略

什么时候必须改写：

- query 包含代词：它、这个、那个、上面的
- query 很短且缺少主体：还有吗？多少钱？怎么办？
- 多轮对话中的追问（非第一轮）

可以用一个简单的规则做预判，减少不必要的 API 调用：

```
/**
 * 判断是否需要 Query 改写
 */
public boolean needsRewrite(String query, List<Message> history) {
    // 第一轮对话且 query 足够长，大概率不需要改写
    if (history.isEmpty() && query.length() > 15) {
        return false;
    }
    // 包含代词，需要改写
    if (query.matches(".*[它它的这个那个这些那些上面].+")) {
        return true;
    }
    // query 太短，大概率省略了上下文
    if (query.length() < 10 && !history.isEmpty()) {
        return true;
    }
    // 有对话历史的情况下，默认都改写（安全起见）
    return !history.isEmpty();
}
```

实际项目中，更稳的做法是：有对话历史就一律改写。改写 API 用小模型（如 Qwen2.5-7B-Instruct），成本很低，但可以避免因为规则没覆盖到而漏改的情况。

Java 实战：Query 改写的实现与效果

1. 基础改写器实现

下面是一个完整可运行的 Query 改写器，基于 Java + OkHttp 调用 SiliconFlow API：

```
public class QueryRewriter {

    private static final String API_URL = "https://api.siliconflow.cn/v1/chat/completions";
    private static final String API_KEY = "you api key";
    // 改用小模型就够了，成本低、速度快
    private static final String MODEL = "Qwen/Qwen2.5-7B-Instruct";
    private static final OkHttpClient client = new OkHttpClient();
    private static final Gson gson = new Gson();

    /**
     * 改写 Prompt 模板
     */
    private static final String REWRITE_PROMPT = """
        你是一个查询改写助手。根据对话历史和用户的最新问题，\
        将问题改写为一个独立的、完整的检索查询。

        要求：
        1. 如果最新问题中包含代词（它、这个、那个等）或省略了关键信息，\
        请结合对话历史补全
        2. 如果问题已经足够完整清晰，请原样输出，不要画蛇添足
        3. 不要添加用户没有提到的信息
        4. 只输出改写后的查询，不要输出任何解释、前缀或多余内容
        5. 改写后的查询应该是一个独立的句子，脱离对话历史也能理解

        对话历史：
        %s

        用户最新问题： %s

        改写后的查询： """;

    /**
     * 执行 Query 改写
     *
     * @param history      对话历史（role + content 的列表）
     * @param currentQuery 用户当前问题
     * @return 改写后的查询
     */
    public static String rewrite(List<Message> history,
                                String currentQuery) throws IOException {

        // 构建对话历史文本
        StringBuilder historyText = new StringBuilder();
        if (history.isEmpty()) {
            historyText.append("（无历史对话）");
        } else {
            for (Message msg : history) {
                String roleName = "user".equals(msg.role) ? "用户" : "助手";
                historyText.append(roleName).append(": ")
                    .append(msg.content).append("\n");
            }
        }

        // 构建改写请求
        JsonObject body = getJsonObject(currentQuery, historyText);

        Request request = new Request.Builder()
            .url(API_URL)
            .addHeader("Authorization", "Bearer " + API_KEY)
```

```

        .addHeader("Content-Type", "application/json")
        .post(RequestBody.create(body.toString(),
            MediaType.parse("application/json")))
        .build();

try (Response response = client.newCall(request).execute()) {
    assert response.body() != null;
    String responseBody = response.body().string();
    JsonObject json = gson.fromJson(responseBody, JsonObject.class);
    return json.getAsJsonArray("choices")
        .get(0).getAsJsonObject()
        .getAsJsonObject("message")
        .get("content").getString().trim();
}
}

private static @NonNull JsonObject getJsonObject(String currentQuery, StringBuilder historyText) {
    String prompt = String.format(REWRITE_PROMPT,
        historyText.toString(), currentQuery);

    JsonObject body = new JsonObject();
    body.addProperty("model", MODEL);
    body.addProperty("temperature", 0.1);
    body.addProperty("max_tokens", 256);
    JsonArray messages = new JsonArray();
    JsonObject userMsg = new JsonObject();
    userMsg.addProperty("role", "user");
    userMsg.addProperty("content", prompt);
    messages.add(userMsg);
    body.add("messages", messages);
    return body;
}

/**
 * 简单的消息数据结构
 */
public static class Message {
    public String role;
    public String content;

    public Message(String role, String content) {
        this.role = role;
        this.content = content;
    }
}

public static void main(String[] args) throws IOException {
    // ===== 场景 1: 指代消解 =====
    System.out.println("===== 场景 1: 指代消解 =====");
    List<Message> history1 = List.of(
        new Message("user", "iPhone 16 Pro 的退货政策是什么? "),
        new Message("assistant",
            "iPhone 16 Pro 因屏幕定制工艺, 拆封后不支持七天无理由退货。" +
            "如有质量问题, 可联系售后处理。");
    );
    String rewritten1 = rewrite(history1, "那它的保修期呢? ");
    System.out.println("原始 query: 那它的保修期呢? ");
    System.out.println("改写结果: " + rewritten1);

    // ===== 场景 2: 上下文补全 =====
    System.out.println("\n===== 场景 2: 上下文补全 =====");
    List<Message> history2 = List.of(
        new Message("user", "iPhone 16 Pro 有什么颜色? "),
        new Message("assistant",

```

```
        "iPhone 16 Pro 有沙漠色钛金属、自然色钛金属、" +
        "白色钛金属和黑色钛金属四种颜色。")

    );
    String rewritten2 = rewrite(history2, "价格呢? ");
    System.out.println("原始 query: 价格呢? ");
    System.out.println("改写结果: " + rewritten2);

    // ===== 场景 3: 口语化转正式 =====
    System.out.println("\n===== 场景 3: 口语化转正式 =====");
    List<Message> history3 = List.of(); // 无历史, 第一轮对话
    String rewritten3 = rewrite(history3, "东西坏了咋整? ");
    System.out.println("原始 query: 东西坏了咋整? ");
    System.out.println("改写结果: " + rewritten3);

    // ===== 场景 4: 已经完整的 query, 不需要改写 =====
    System.out.println("\n===== 场景 4: 不需要改写 =====");
    List<Message> history4 = List.of();
    String rewritten4 = rewrite(history4,
        "iPhone 16 Pro 的退货政策是什么? ");
    System.out.println("原始 query: iPhone 16 Pro 的退货政策是什么? ");
    System.out.println("改写结果: " + rewritten4);
    }
}
```

2. 改写前后的效果对比

运行结果（示意）：

```
===== 场景 1: 指代消解 =====
原始 query: 那它的保修期呢?
改写结果: iPhone 16 Pro 的保修期是多久?

===== 场景 2: 上下文补全 =====
原始 query: 价格呢?
改写结果: iPhone 16 Pro 的价格是多少?

===== 场景 3: 口语化转正式 =====
原始 query: 东西坏了咋整?
改写结果: 东西坏了怎么修理?

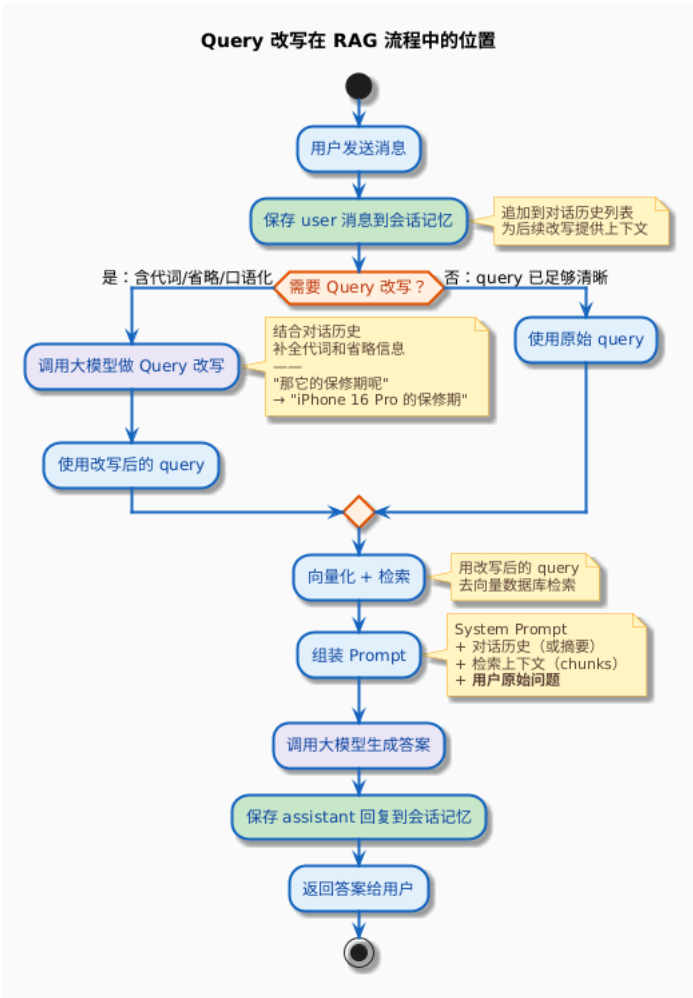
===== 场景 4: 不需要改写 =====
原始 query: iPhone 16 Pro 的退货政策是什么?
改写结果: iPhone 16 Pro的退货政策是什么?
```

几个关键观察：

- 场景1**：代词“它”被成功替换成了 iPhone 16 Pro，检索系统现在能精准命中 iPhone 16 Pro 保修相关的 chunk
- 场景2**：省略的主体被补全了，“价格呢”不再是一个孤立的、无法检索的 query
- 场景3**：即使没有对话历史，口语化的表达也被转化成了更适合检索的正式表达
- 场景4**：query 本身已经完整清晰，模型原样输出，没有画蛇添足——这一点很重要，过度改写比不改写更危险

3. 改写在 RAG 流程中的位置

把 Query 改写加入后，多轮对话 RAG 的完整流程是这样的：



注意两个细节：

1. **检索用改写后的query，但Prompt里放的是用户原始问题。**改写的目的是让检索更精准，不是改变用户的问题。模型生成答案时，应该针对用户的原始问题回答，而不是改写后的 query。保险点改写问题前后放进去也可以。
2. **Query改写在会话记忆读取之后、检索之前。**这个位置很关键——改写需要对话历史作为输入，改写的结果用于检索

生产环境的注意事项

1. 改写质量的监控

Query 改写是一个容易默默出错的环节——改写结果不好，检索不到相关内容，模型给出一个兜底回答或者答非所问，用户可能只是觉得这个 AI 不够聪明，不会意识到是改写环节出了问题。

建议记录每次改写的完整信息：

```
{
  "session_id": "session-001",
  "original_query": "那它的保修期呢？",
  "rewritten_query": "iPhone 16 Pro 的保修期是多久？",
  "history_length": 2,
  "rewrite_latency_ms": 320,
  "timestamp": "2025-03-07T10:30:00Z"
}
```

定期抽检改写日志，关注几个指标：

- 改写率：**所有请求中触发了改写的比例。如果太低，可能是判断规则太严格，该改写的没改写
- 过度改写率：**人工标注后发现模型画蛇添足的比例。如果太高，需要调整 Prompt
- 改写后检索提升率：**对比改写前后的检索命中率（需要配合人工标注或自动化评测）

2. 改写失败的兜底

改写 API 可能因为网络超时、模型服务不可用、返回格式异常等原因失败。这时候应该用原始 query 兜底，而不是报错。

```
public String safeRewrite(List<Message> history, String query) {
    try {
        String rewritten = rewrite(history, query);
        // 基本校验: 改写结果不能为空, 不能太长
        if (rewritten != null && !rewritten.isEmpty()
            && rewritten.length() < 500) {
            return rewritten;
        }
    } catch (Exception e) {
        log.warn("Query 改写失败, 使用原始 query: {}", e.getMessage());
    }
    return query; // 兜底: 返回原始 query
}
```

Query 改写是锦上添花，不是雪中送炭。即使改写失败，用原始 query 检索也能有一定的效果（只是精度可能差一些）。千万不要因为改写失败就让整个 RAG 流程挂掉。

3. 改写缓存

同一个 session 内，用户可能重复提问或者问类似的问题。可以用一个简单的缓存避免重复调用改写 API：

```
// 缓存 key = sessionId + 原始 query 的哈希
Map<String, String> rewriteCache = new ConcurrentHashMap<>();

public String rewriteWithCache(String sessionId, List<Message> history,
                               String query) throws IOException {
    String cacheKey = sessionId + ":" + query.hashCode();
    return rewriteCache.computeIfAbsent(cacheKey,
        k -> {
            try {
                return rewrite(history, query);
            } catch (IOException e) {
                return query;
            }
        });
}
```

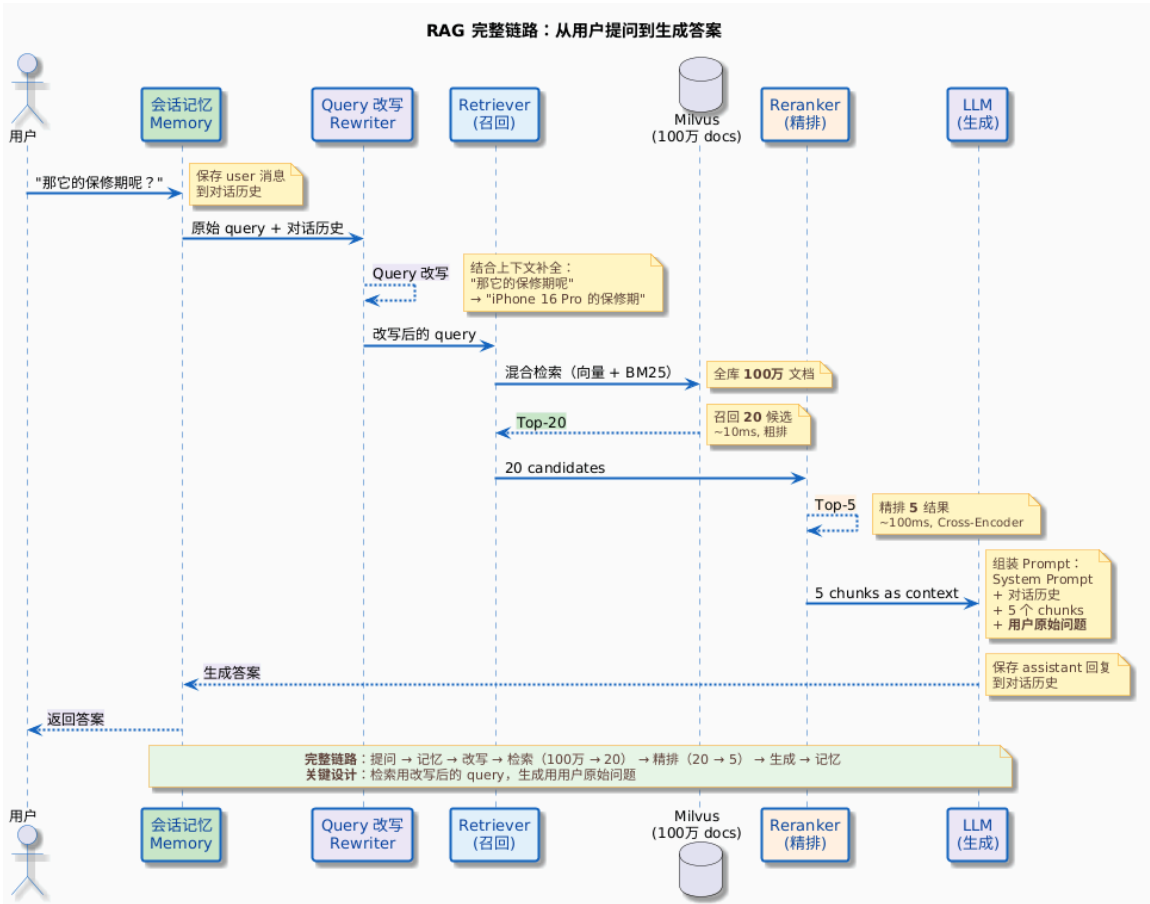
注意：缓存的粒度要包含 sessionId，因为同样的 query 在不同的对话上下文中，改写结果可能不同。“价格呢？”在聊 iPhone 时改写成 iPhone 的价格，在聊 AirPods 时改写成 AirPods 的价格。

小结与下一篇预告

这篇讲了 Query 改写，核心要点回顾：

- 检索系统的失忆问题**：会话记忆让模型有了上下文理解能力，但检索系统拿到的还是用户的原始 query，包含代词和省略信息的 query 检索效果很差
- 五种改写策略**：指代消解（必做）、上下文补全（必做）、口语化转正式（推荐）、多意图拆分（按需）、关键词扩展（混合检索场景有用）
- 用大模型做改写**：一个 Prompt 覆盖大部分改写场景，用小模型即可，成本低延迟小
- Prompt设计是关键**：要明确告诉模型不要画蛇添足，避免过度改写
- 改写在RAG流程中的位置**：会话记忆之后、检索之前。检索用改写后的 query，生成回答用用户原始问题

结合前面几篇，多轮对话 RAG 的完整链路已经清晰了：



到这里，RAG 系统已经具备了完整的多轮对话能力。但还有一个问题没有解决：用户发来一条消息，系统怎么知道应该去检索知识库，还是去调用工具，还是直接闲聊？

比如用户说“今天天气怎么样”，这明显不是一个知识库检索的问题；用户说“帮我查一下我的年假余额”，这应该调 Function Call；用户说“iPhone 16 Pro 的退货政策是什么”，这才需要走 RAG 检索。

下一篇咱们来聊 **意图识别与问题路由** ——让 RAG 系统在收到用户消息后，自动判断应该走哪条路：知识库检索、工具调用、还是直接对话，做出正确的路由决策。